Project Status and Work Split

Resource-Aware Multi-GPU Semantic Parallelism for Planned Diffusion (CSC 548)

Khush Patel and Rishabh Ravi Kumar · Status as of November 2, 2026 · Repository commit 3d5d269

Summary

The CPU-side groundwork is in place. The baseline is repaired and records timing and memory. Each planning stage is now an explicit list of chunk tasks. The C++ round-robin and LPT schedulers are callable from Python. A threaded local runtime assigns chunks, runs them, gathers results and puts them back in order. With the C++ extension built, all 67 tests pass.

Nothing runs on more than one GPU yet. The remaining work is the GPU half of the proposal: keeping position IDs when chunks are split, the single-GPU correctness check, the `torchrun` runtime, the adaptive 1/2/4-GPU scheduler and the experiments. The position-ID fix is the critical path, because every multi-GPU result depends on it.

4 proposal items done	**2** partly done	**7** not started

1. Changes made so far

Area	What changed	Main files	Commits
Baseline CLI	Fixed the undefined `--use_fp16` argument and the model not being moved to the GPU. Added `--metrics_json`: latency measured with CUDA synchronization, peak GPU memory, tokens/s, and per-stage plan text, chunk lengths and step counts.	`eval/pd_generate.py`	593509b, aaa 0105–c8a58c5
Stage extraction	Added `ChunkTask` and `PlannedStage`. The non-cached generation loop builds one `PlannedStage` per planning stage and runs diffusion from it. Output is meant to be unchanged; this has not been checked yet.	`semantic_parallel/stage.py` `dream/generation_utils.py`	461ab4d
C++ scheduler	Round-robin and LPT (longest chunk to least-loaded worker, deterministic tie-breaking), input validation, pybind11 bindings, `setup.py` build and a Python adapter returning `WorkerAssignment`.	`scheduler/native/*` `scheduler/adapter.py`	461ab4d, 1f8ad4a
Runtime contract	Assignment validation (contiguous worker IDs, every chunk assigned exactly once), reassembly of results in `chunk_id` order, and a thread-per-worker local runtime that reports compute and idle time per worker.	`distributed/runtime.py` `distributed/local_runtime.py`	d31620e
Simulation	Sleep-based fake executor (time proportional to chunk length) and a driver that reports sequential time and measured speedup for any policy and worker count.	`profiling/simulation.py`	d31620e
Tests and docs	Tests for stage extraction, both schedulers, runtime helpers, the local runtime and the simulation, using the proposal's A=120 / B=80 / C=35 / D=20 example on 1, 2 and 4 workers. README now has a project status section.	`tests/` `README.md`	3d5d269 and earlier

Test status: 38 pass and 10 are skipped when the C++ extension is not built; 67 of 67 pass once it is built with `pip install -e .`

2. Proposal checklist

Proposal item	Status	Notes
Keep upstream Python/PyTorch Planned Diffusion and Dream-7B	Done	CLI fixed and instrumented. No GPU baseline results are committed yet.
C++ scheduling module exposed through pybind11	Done	Round-robin and LPT, both tested.
Scheduler reads the plan's chunk metadata	Done	Non-cached path only. Output-equivalence check still outstanding.
First policy: longest chunk to least-loaded GPU (LPT)	Done	Includes the A/B/C/D example.
Assign chunks to GPU workers	Partial	Works on CPU threads with a fake executor. No GPU runs anything yet.
(i) Single-GPU baseline measurement	Partial	Code ready; needs a measured run on ARC.
Coordinate synchronization points (gather at <sync/>)	Not started	No torch.distributed code yet.
Preserve logical position IDs across devices	Not started	Dream overwrites position_ids with arange (dream/modeling_dream.py:763). Blocks all GPU work.
(ii) Static multi-GPU assignment	Not started	Scheduling exists; GPU execution does not.
(iii) Workload-aware scheduler choosing 1, 2 or 4 GPUs	Not started	The project's main contribution.
Profile real execution and fit a cost model	Not started	Inputs: chunk length, prefix length, steps.
Evaluation sweep and metrics (latency, S_p, idle, overhead, memory, GPU-seconds)	Not started	Simulation reports idle time and speedup, but from sleep timers only.
Test the granularity-threshold hypothesis	Not started	Depends on everything above.

3. Issues to settle before GPU work

1. **Position IDs are discarded.** `DreamBaseModel.forward` rebuilds `position_ids` with `arange`, so a chunk run without the chunks before it gets the wrong rotary positions and produces different text. Fix and test this first.
2. **Workers should run their chunks together, not one after another.** The local runtime runs a worker's chunks in sequence, and the scheduler's cost is the sum of chunk lengths. On a GPU, a worker given C(35) and D(20) should run them in one sparse-mask sequence: 35 forward passes instead of 55. A worker's cost is roughly `steps(longest chunk) × forward_time(prefix + total chunk length)`.
3. **The speedup may be capped by forward-pass time.** With A=120 on its own GPU, the critical GPU still runs 120 steps, so the gain is about `forward_time(prefix+255) / forward_time(prefix+120)`. For a 7B model at a few hundred tokens this ratio may be close to 1. Measure it in week 1; the result is the basis for the threshold hypothesis.
4. **Scope limits for the first version.** Non-cached path only (the cached path has no stage extraction). Reject `--disable_block_sparsity` in multi-GPU mode, because chunks are no longer independent there. Use temperature 0 for all correctness checks.
5. **Housekeeping.** Seven compiled `.pyc` files are tracked even though `.gitignore` excludes `__pycache__/`; remove them with `git rm --cached`. Build the C++ extension in the shared ARC environment so scheduler tests stop skipping.

4. Interface to agree on first

Fixing this one function signature lets both tracks work at the same time. Rishabh implements it; Khush's runtime calls it, using a fake with the same signature until the real one is ready.

```
def execute_chunks(model, stage: PlannedStage, chunk_ids: list[int]) -> dict[int, torch.Tensor]:
    # Runs the stage prefix plus these chunks as one sequence with the sparse mask,
    # their original position IDs, and the stage's diffusion step count.
    # Returns each chunk's generated tokens, keyed by chunk_id.
```

Also agree on one metrics record for every run: policy, GPU count, stage ID, chunk lengths, prefix length, diffusion steps, per-rank compute and wait time, scheduling time, collective time, end-to-end latency, peak memory per rank and GPU-seconds.

5. Work split

Khush wrote the scheduler and runtime, so Khush continues on that side. Rishabh takes the model side, which is the critical path. Each task lists what "done" means so the hand-offs are clear.

Rishabh: model and measurement track

ID	Task	Done when	Week
R1	Profile on ARC: single-GPU baseline (T_1) on a fixed prompt set; Dream-7B forward time against sequence length (128 to 2048 tokens); chunk counts, chunk lengths, prefix lengths and stage counts for about 50 AlpacaEval prompts.	CSVs committed, plus a short note on the expected speedup ceiling	1
R2	Make `DreamBaseModel.forward` use caller-supplied `position_ids` on the non-cached path.	Combined generation is token-identical before and after the change	1
R3	Implement `execute_chunks`: prefix plus a subset of chunks, the matching part of the sparse mask, original position IDs and the stage's step count.	Runs on one GPU for one and two chunks	2
R4	Correctness check (Checkpoint 3): compare combined vs. per-chunk logits and tokens at temperature 0, for one chunk and for two unequal chunks.	Test passes, or differences are explained and bounded. Go/no-go for multi-GPU	2
R5	Fit the cost model from R1 and Checkpoint 4 timings: stage time against steps, prefix length and total chunk length.	Coefficients and fit error handed to Khush for K4	3
R6	Multi-stage, model side: after each gather, rebuild the sequence and attention mask, restore <sync/> and continue planning.	Full multi-stage responses match single-GPU output at temperature 0	4
R7	Report sections: background, correctness method, cost model.	Draft in shared report	5

Khush: scheduler, runtime and experiments track

ID	Task	Done when	Week
K1	Make the worker executor take a worker's whole chunk list. Update the local runtime, simulation and tests. Change the scheduler's cost to `steps(longest) × f(total length)`.	Updated tests pass	1
K2	`torch.distributed` backend: rank 0 plans and schedules, broadcasts prefix tokens and task metadata; ranks call `execute_chunks`; tokens are gathered by `chunk_id` and rank 0 rebuilds the sequence. Fail fast with timeouts. Develop with `gloo` and the fake executor on CPU.	2- and 4-process CPU tests pass, including a deliberate rank failure	1–2
K3	One stage with real Dream on 2 and 4 GPUs (Checkpoint 4), using R3. Reject later sync stages with a clear error.	Output matches single-GPU; per-rank timings recorded	3

ID	Task	Done when	Week
K4	Adaptive scheduler in C++: cost-aware LPT using the R5 model, plus `choose_worker_count(tasks, max_gpus)` that picks 1, 2 or 4 GPUs. pybind11 bindings and tests.	Chooses fewer GPUs when the predicted gain is below the overhead	3–4
K5	Experiment harness: sweeps over GPU count, chunk count, response length and imbalance (synthetic plans allowed), writing the agreed metrics record.	One command produces all result CSVs	4
K6	Report sections: system design, scheduler, results.	Draft in shared report	5

Shared

Housekeeping and the ARC environment (week 1); Checkpoint 4 integration (week 3); Checkpoint 5 integration (week 4); running the experiments, plots and final presentation (week 5).

6. Timeline

Week	Rishabh	Khush	Milestone
1 · Nov 2–8	R1, R2	K1, start K2	Profiling numbers in hand; position-ID fix merged; housekeeping done
2 · Nov 9–15	R3, R4	Finish K2	Correctness check passed; distributed backend works on CPU
3 · Nov 16–22	R5; support K3	K3, start K4	One stage on 2 and 4 GPUs: comparison (ii)
4 · Nov 23–29	R6	K4, K5	Multi-stage on GPUs; adaptive scheduler: comparison (iii). Thanksgiving week, plan lighter
5 · Nov 30–Dec 6	Experiments, plots, R7	Experiments, plots, K6	Final results and report draft

Dependencies. R2, R3 and R4 must be done before K3 uses real Dream. K2 needs no GPU and runs in parallel. R1 feeds R5, which feeds K4. The timeline assumes the report is due in early December; shift it if the deadline differs.